PratiBimb

प्रतिबिम्ब — a reflection that reveals the structure, never the identity.

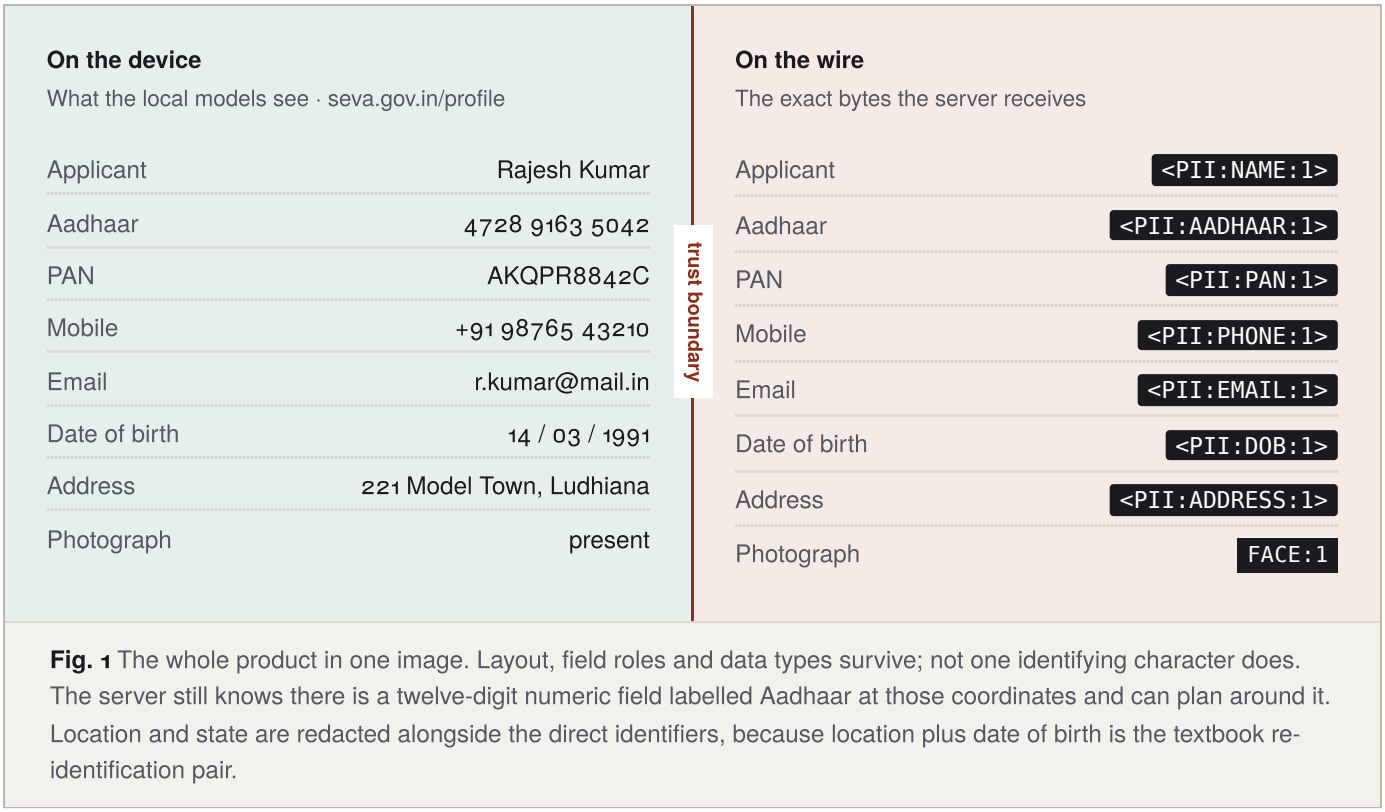

Fig. 1 The whole product in one image. Layout, field roles and data types survive; not one identifying character does. The server still knows there is a twelve-digit numeric field labelled Aadhaar at those coordinates and can plan around it. Location and state are redacted alongside the direct identifiers, because location plus date of birth is the textbook re-identification pair.

Document
Pre-product engineering dossier

Version
4.0 — final, supersedes v3.0

Audience
Build team, evaluators, finale panel

Status
Implementation baseline. Contracts frozen; models replaceable behind them; every model gated on a feasibility row.

01 · The brief

What we were asked to build

ISRO wants a browser agent that can see your screen and act on it, without your screen ever reaching their server.

Agentic pipelines run server-side, which limits what a user can safely share with them. Deploying a local agent in the browser removes the need to send sensitive data at all — only non-sensitive information, such as screen structure and application fields, need reach a server.

Participants must build a privacy-preserving vision agent that runs in the browser. A local Vision Transformer or equivalent model reads the screen and decides on that basis. If visual context must be sent to a server, the client sanitizes sensitive and personally identifying data first — using DOM tags or any other method — before any network request is made. It should dynamically detect and redact sensitive elements: blurring faces, blacking out passwords, masking PII. Only anonymized, unidentifiable data may be transmitted. The server must be aware of the redaction scheme and process data accordingly, returning actionable commands for the browser agent to execute.

Deliverables named in the statement

- Client extension running in Chrome and Firefox, with a vision model executing locally, e.g. via WebGPU.
- A privacy filter — bounding-box redaction, semantic obfuscation, or masking — demonstrated clearly.
- A server hosting an open-weight LLM or VLM that interprets sanitized context and returns either processed data or a UI action for the client to execute.
- One end-to-end task that visibly assists a user.

Where the marks are

The rubric is the specification. Every decision in this document is traceable to a row in it.

Fig. 2 Engineering effort splits roughly 70/30 in favour of the client. **Corrected in v3.0:** v2.0 claimed 85% was decided client-side. Visual-context accuracy is measured end-to-end and depends on the server's ability to reason over a redacted frame, so it is shared, not ours alone. Overstating our share is the kind of claim a panel will test — and the honest number is still overwhelming.

02 · Product thesis

A privacy firewall that happens to power an agent

Redact the value, preserve its semantic type, and let only the trusted client restore it.

The obvious build is screenshot → blur faces → send to a VLM. It fails, and it fails in a way that is easy to demonstrate. The moment you black out the user's phone number, the server can no longer instruct the agent to type the phone number. Privacy has eaten the product.

PratiBimb resolves this with a redaction manifest and a memory-only vault. Every sensitive span is replaced by a typed placeholder — `<PII:PHONE:1>` — that carries the shape of the data without its content. The server plans in placeholders. The client substitutes real values at the instant of execution. The server is functionally complete and structurally blind.

Five principles

1. **The client never trusts the server.** Server output is a proposal, validated against an allowlist locally, and re-validated against the live page, before anything executes.
2. **Fail closed.** When a detector errors, times out, or disagrees, the union of masks wins. An over-mask costs nothing, because re-hydration restores the capability. A missed Aadhaar number ends the demo.
3. **Page content is data, never instruction.** Text scraped from a website cannot direct the agent. Only the user's stated goal can.
4. **Measure before optimising.** The evaluation harness is built in week two, not week six.
5. **Secrets cross as references; everything else crosses as itself.** v4.0 The server may reference a vault token, and it may propose a plain literal for a field that holds nothing sensitive — a search term, a district, a year. What it may never do is emit a string the vault already contains, because that would mean it reconstructed a secret it was never given. The client decides which of those three cases it is looking at, on every action.

03 · Threat model new in v3.0

Four adversaries, four answers

A privacy claim without a named adversary is decoration. These are the four we defend against, and where in the boundary each defence sits.

Adversary	What it can do	Mitigation, and where it lives
Compromised or curious server	Read every byte we transmit; return a malicious plan	Values never leave: opaque fill on the outgoing bitmap, tokens in the manifest, vault in client RAM. Returned plans pass the action allowlist and the freshness check before touching the page. *Egress guard + local validator.*
Malicious webpage	Inject instructions into text the agent scrapes; overlay or clickjack a control	Page text is framed as data in the prompt and never as instruction. DOM/vision disagreement above threshold flags an overlay. Irreversible actions stop for a human. *Content script + prompt construction + confirmation tier.*
Faulty local detector	Miss an Aadhaar number, time out, or crash mid-pipeline	Four-channel union with fail-closed semantics; a timeout counts as a positive. Differential second-pass verifier plus value-aware residual check. Three failures block the request rather than degrading it. *Privacy verifier.*
Incorrect or hostile VLM	Hallucinate coordinates, invent tokens, emit an unsafe action, echo a value	Guided JSON constrains generation; Pydantic validates on egress; the client re-validates on ingress. Unknown tokens abort. A literal aimed at a redacted field aborts. A literal that matches a vault value is treated not as a defect but as a leak: the session halts and the run is recorded as failed. *Action schema + validator.*

Stated scope limit — prepare this answer

The server sees layout, field roles, data types and the user's goal. A determined adversary correlating quasi-identifiers across many sessions is outside our threat model, and we say so rather than implying otherwise. We reduce the surface where it is cheap: coarse location, employer and institution names are redacted alongside direct identifiers, because location plus date of birth is the classic re-identification pair. We do not claim k-anonymity or differential privacy, and we will not let a slide imply we do.

04 · The loop

Observe, sanitize, reason, act

One pipeline, memorised by everyone on the team, printed on every slide.

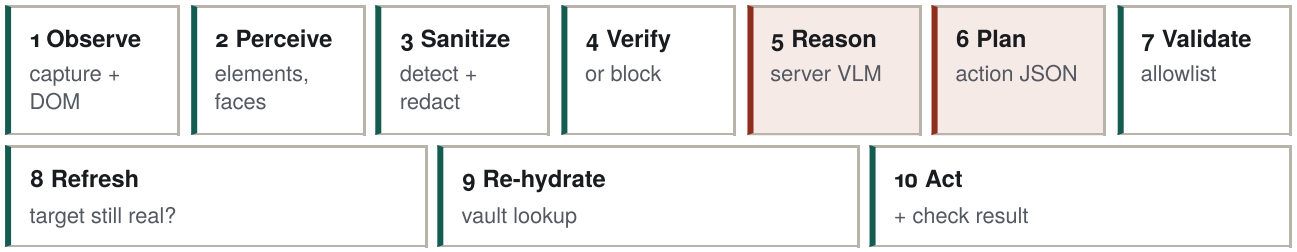

Fig. 3 Six local stages surround the server call, and they are the entire contribution. A conventional browser agent collapses 1–4 into "take a screenshot" and 7–10 into "click the coordinates". **Added in v3.0:** stage 8, action freshness. A plan computed against a frame that is now 900 ms old may target an element that has moved, closed, or been replaced by an overlay.

Stage 8 in detail: action freshness

Before executing, the client re-checks that the target element still exists, that its role and accessible name still match what was reported to the server, that it is visible and enabled, and that its bounding box has not moved beyond a tolerance. A failed freshness check does not guess — it discards the plan, re-observes, and issues a new request. This is cheap (single-digit milliseconds), and it closes the window in which a page can swap a benign control for a harmful one after the screenshot but before the click.

---

05 · System

Architecture

Two processes, one boundary, one choke point for everything that crosses it.

Fig. 4 The tripwire is deliberate defence in depth: an independent, adversarial measurement of the client's redaction quality. "Server-side PII detections across the full evaluation run: zero" is a stronger claim than any self-report from the client. **Changed in v3.0:** the tripwire logs metadata only. A privacy component that writes leaked plaintext into a server log has recreated the problem it exists to detect.

The egress invariant

This is stated as a security property, not a coding convention, because it is the one claim the whole submission rests on:

> **Invariant E**
>
> No outbound network request originates from the extension unless it was issued by the egress module, and the egress module issues no request unless the verifier has returned `verified === true` for a byte artifact whose hash equals the hash of the buffer being transmitted.
>
> Enforced four ways: a lint rule that fails the build on `fetch`, `XMLHttpRequest`, `sendBeacon` or `WebSocket` outside the egress module; a Playwright test that installs a request interceptor and asserts zero requests under each injected failure; a manifest content security policy restricting `connect-src` to the configured server origin; and the hash pin described immediately below.

One payload, one hash, no window in between new in v4.0

Verifying a canvas and then encoding it, or verifying an object and then serializing it, leaves a gap in which the thing checked is not the thing sent. The payload is therefore constructed once, as a single immutable artifact: the encoded WebP frame and the serialized manifest, assembled into the exact multipart body that will go on the wire. That artifact is hashed. Verification runs against it. The egress module accepts a buffer only when its hash equals the hash the verifier signed.

Nothing is re-encoded, re-serialized or mutated between those two moments — a change of a single byte invalidates the pin and the send fails closed. This also makes the audit trail exact: the hash the privacy ledger displays is the same number the verifier approved and the same body the panel can read in the network inspector. Three independent views of one artifact, and they either agree or the request never happened.

The coordinate contract

Twenty-five percent of the score is grounding accuracy, and the commonest way to lose it is silently: everything works at device pixel ratio 1.0 on the development laptop and falls apart on a HiDPI judging machine at 125% zoom. One canonical space is defined and every component converts at its edge.

Space	Used by	Conversion rule
CSS viewport pixels — canonical	Manifest, action plans, ledger	Origin at viewport top-left. All manifest boxes and all returned action coordinates are in this space, always.
Device pixels	`tabs.captureVisibleTab` output	Divide by `devicePixelRatio` on capture. Recorded in the manifest `capture` block.
Capture pixels	Downscaled frame sent to the server	Uniform scale factor recorded in the manifest. The server may reply in either capture or CSS space; the field is explicit and the client converts.
Document pixels	Off-screen elements known from DOM	Offset by scroll position. Never sent as an action target without a preceding scroll.

A CI test renders a fixture at DPR 1.0, 1.5 and 2.0 and at 100% and 125% zoom, and asserts that the same logical element resolves to the same CSS-pixel box in all six. This test exists before the executor is written.

Content below the fold new in v3.0

Capture is viewport-only, so on a long form the agent cannot see most of the fields. The DOM extractor therefore reports off-screen elements as *known but uncaptured*: role, accessible name and document-space geometry, with no pixel evidence and no vision cross-check. The server can plan a `scroll` toward a named field it has never seen, and the next observation confirms it. Without this, a multi-field form silently caps the agent at whatever fits on one screen — which is exactly the demo the panel will ask us to run.

06 · Perception

Vision on every changed frame, heavy analysis only when it earns it

Four models, roughly 120 MB resident. The expensive tiers are gated. The vision tier is not.

The framing correction that matters most in v3.0

Version 2.0 presented tiering as evidence that PratiBimb rarely runs vision — the local VLM was shown firing zero times, and OCR on nought to two regions. That is good engineering and a bad answer to this brief. The problem statement requires a local vision model that reads the screen and decides on that basis. A panel that watches a run where the DOM does all the work has an easy line: *you built a DOM scraper with a redaction layer.*

The architecture does not change. The framing and the demonstration do. T1 vision runs on every frame where the screen changed — that is the always-on perception tier. To is a debounce, not an avoidance strategy. And the demonstration now includes content where the DOM is empty by construction, so vision is visibly load-bearing rather than merely present.

Tiered perception over a ten-step form-filling task

T0	Change gate — structural signal plus bounded visual polling Sub-millisecond for the structural half, no model. MutationObserver names the dirty subtree exactly and for free; registered dynamic regions are hashed at a bounded rate; a low-rate full-frame hash bounds the worst case. See the change policy below.	~100 evaluations ~8 captures
T1	Perceive — UI element detector + YuNet faces 12.3 MB. Runs on every *changed frame* — precisely defined below, and not the same thing as every rendered frame. This is the local vision model the brief requires, and it is on the normal path.	fires ~8×
T2	Sanitize — selective OCR + GLiNER-PII 105 MB. Runs when a request is about to leave, on non-DOM regions and dirty subtrees only.	fires ~3×
T3	Local VLM — offline fallback ~240 MB, loaded on demand in an isolated worker, torn down immediately after. Kept out of the normal path so the resource figure stays defensible.	fires 0× online

Fig. 5 Reported under client resource utilisation: peak heap, peak GPU memory, weights on disk, and this firing distribution. The capture count is separated from the evaluation count because `tabs.captureVisibleTab` is neither free nor unrestricted — it is rate-limited, particularly under `activeTab` — so a change gate that assumed a free capture per evaluation, as v2.0 did, was not implementable.

Two change signals, one policy new in v4.0

Version 3.0 made MutationObserver the primary gate and left the impression that it reports everything that visually changed. It does not, and the claim would not survive a question. A page can change on screen with no mutation record at all: CSS animations and transitions, canvas and WebGL drawing, video frames, pseudo-element content, timer- and scroll-driven visual state, and repaints inside cross-origin frames.

The precise statement is that **MutationObserver is the structural signal, and perceptual hashing is the visual signal for regions structural observation cannot reach.** A changed

frame is one in which the change policy reports an observable-state change from either signal — that is the definition T1 is gated on.

Signal	Mechanism	Covers	Cost
Structural	MutationObserver on the document; ResizeObserver on tracked elements	Subtree edits, attribute and text changes, geometry, insertion and removal	Free, event-driven, names the dirty node
Visual	dHash over registered dynamic regions, at a bounded poll rate, only while they intersect the viewport	canvas, video, WebGL, elements with running animations	One partial capture per poll, not per rendered frame
Safety net	Full-frame dHash at a low fixed interval	Anything both signals miss	One capture per interval, capped

Dynamic regions are enumerated rather than guessed: `<canvas>` and `<video>` elements, plus anything reporting a running animation through `document.getAnimations()`. The enumeration re-runs on structural change, not per frame, and `IntersectionObserver` suppresses polling for regions that are off screen.

> **The honest answer to "how do you know a frame changed without capturing continuously?"**
> We do not claim to detect every visual change for free. We claim three narrower things, and each is testable. Structural change is observed exactly, at zero cost, and identifies which node changed. Visually dynamic regions are enumerated once and polled at a bounded rate, so cost scales with how dynamic the page actually is rather than with its frame rate. And a low-rate full-frame hash bounds the worst case, so nothing can go unnoticed indefinitely.
>
> On a page that is one full-screen canvas animation, this degrades to the poll rate, and we will say so rather than pretend otherwise. Every page in the demonstration and the evaluation set is measured with the tier firing distribution visible, so the cost of the policy is a reported number, not an assurance.

The DOM is the primary sensor, and vision is not decorative

The brief explicitly permits sanitisation "using DOM tags or any other method". For text the browser rendered itself, you already hold the exact string and its bounding box — asking OCR to rediscover it is slower and less accurate. But the DOM is blind to an entire class of content that appears constantly on Indian government and banking pages: scanned identity documents, canvas-rendered charts, embedded PDFs, and photographs.

The DOM already knows

— role, accessible name, label
— input type, autocomplete token

Only vision can see

— scanned PAN and Aadhaar cards
— text inside images and PDFs

Fused element graph

— matched on IoU > 0.5

— exact text and its box
— visibility, enabled state, ARIA
— free, exact, no inference error

— canvas-rendered interfaces
— faces, signatures, stamps
— cross-origin iframes, overlays

— DOM-matched: actioned by selector, robust to reflow
— vision-only: actioned by coordinate, synthetic id
— disagreement above threshold flags an overlay

Fusion raises grounding accuracy and cuts latency at the same time, so it scores on the visual-context metric and the latency metric together. It also catches disagreements between DOM and pixels — the signature of an overlay or a clickjacking attempt, which is why the same mechanism appears in the threat model.

Precise terminology v3.0

Version 2.0 described the content script as reading the accessibility tree. It does not, and cannot: `chrome.automation` is ChromeOS-only for extensions, and no content-script API exposes the browser's native AX tree. What we build is a *derived element graph* from roles, ARIA attributes, accessible-name computation, computed styles and geometry. The approximation is entirely adequate. Claiming the real thing to a panel that knows the extension APIs is an unforced credibility loss, of exactly the kind the WebGPU section below warns against.

---

07 · Privacy

Four detectors, one union, no exceptions

Recall is prioritised over precision, because over-masking is free and under-masking is fatal.

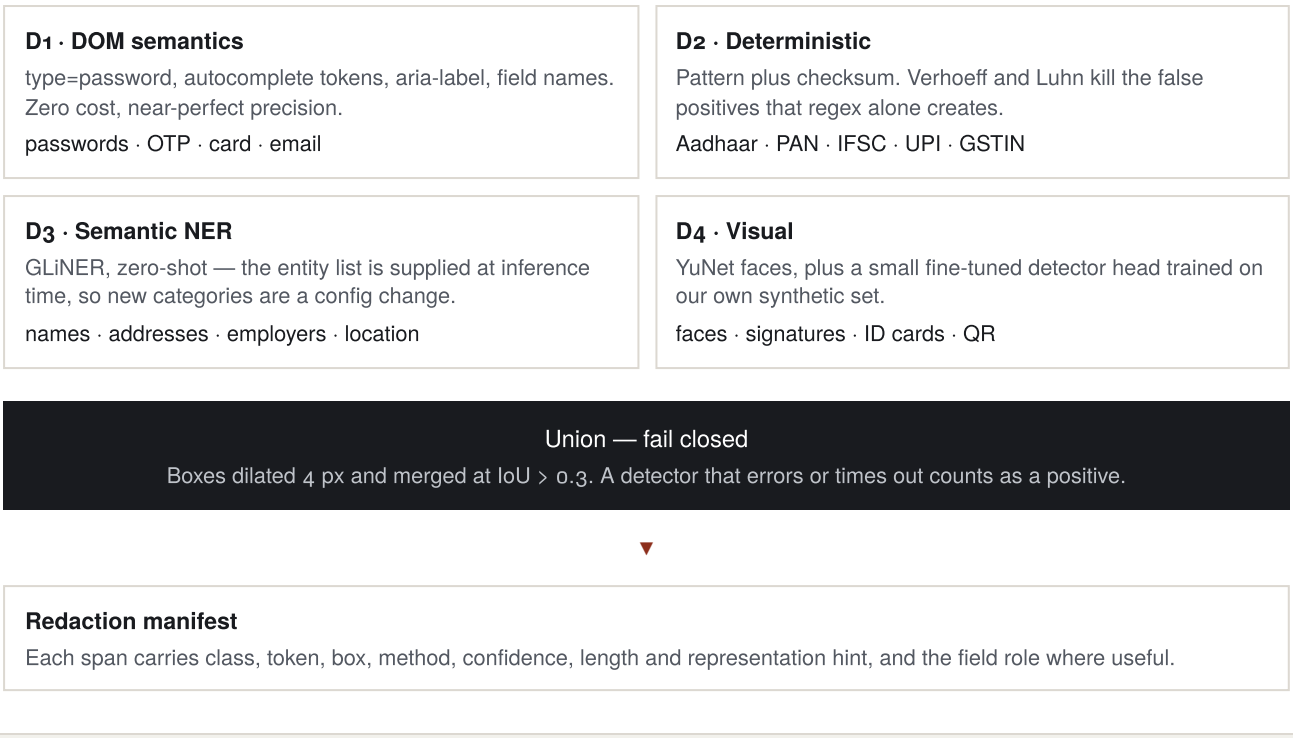

Fig. 6 The four channels are ordered by confidence, not by sophistication. D1 and D2 need no machine learning at all and already cover most structured Indian PII — which is why they ship first and the pipeline can be built and measured before any model is integrated.

Indian identifier validators

This is where the submission earns its domain credibility. Format matching alone flags every twelve-digit order number as an Aadhaar; the checksum is what makes precision defensible, and precision is a fifth of the score.

Identifier	Pattern	Validation that earns the precision
Aadhaar	`\d{4}\s?\d{4}\s?\d{4}`	Verhoeff checksum over all twelve digits
PAN	`[A-Z]{5}\d{4}[A-Z]`	Fourth character must be a valid holder-type code. Note: PAN's final character is a check digit whose algorithm is not public, so this is format plus structure, not a checksum — do not oversell it.
IFSC	`[A-Z]{4}0[A-Z0-9]{6}`	Fifth character is always zero; bank-code table lookup
UPI VPA	`[\w.\-]{2,}@[a-z]{2,}`	Handle allowlist — okaxis, ybl, paytm, upi
Mobile	`(\+91[-\s]?)?[6-9]\d{9}`	Leading digit constrained to 6–9
Payment card	`13–19 digits`	Luhn checksum plus issuer range
GSTIN	`\d{2}[A-Z]{5}\d{4}[A-Z]\d[Z][A-Z\d]`	State-code table, embedded PAN check, and the mod-36 check digit over the first fourteen characters **v3.0**
Vehicle registration	`[A-Z]{2}\d{2}[A-Z]{1,3}\d{4}`	State and RTO code tables

Confidentiality classes, kept as configuration **v3.0**

The recommendation to extend beyond obvious PII is right in principle and dangerous in scope. A configurable policy engine is six weeks of work for zero rubric points. Because GLiNER is zero-shot, the same benefit is available as a config file:

Class	Examples	Treatment
CRITICAL	passwords, OTP, card numbers, private keys	Masked; never tokenised for server reference; never re-hydrated without explicit per-use confirmation
SENSITIVE	Aadhaar, PAN, GSTIN, account numbers, medical text	Masked and tokenised; re-hydrated at execution
PERSONAL	name, phone, email, address, DOB, photograph, employer, city	Masked and tokenised; re-hydrated at execution
CONFIDENTIAL	balances, internal document titles, source code, salary figures	Masked and tokenised. Entity list lives in a JSON config passed to GLiNER at inference time — adding a category is a one-line change, not a training run.
PUBLIC	navigation, headings, button labels, generic body copy	Transmitted unchanged; this is what lets the server reason at all

Redaction, and why it is opaque fill

Gaussian blur and pixelation are lossy but recoverable in distribution — deconvolution and reconstruction attacks routinely recover blurred text and mosaiced faces when the kernel can be estimated. Applying a CSS filter and then screenshotting is worse still: it is a compositing race, and if the frame has not flushed, raw pixels ship.

Every transmitted mask is therefore a constant-colour opaque fill, composited onto an OffscreenCanvas inside the worker. The un-redacted bitmap is closed immediately after masking and never referenced again. Blur remains available as a display-side courtesy, never on the buffer that leaves.

Semantic labels on top of the fill v3.0

The token string is rendered in small type over the opaque fill before encoding, so the outgoing image carries `<PII:AADHAAR:1>` where the digits were. The server then sees the same token in the pixels and in the manifest, which measurably helps a VLM associate a field with its type and its label. Two constraints: the label is composited strictly after the fill, never instead of it; and the verifier is aware of its own labels and does not flag them as residual text.

The verifier, and why the second pass must differ

> **The tautology v2.0 shipped with**
>
> Re-running the same detectors at the same settings on the same image will find the same nothing they found the first time. A detector that missed an Aadhaar number will miss it again. As specified in v2.0, the verifier could only ever confirm its own first pass.

Verification is therefore explicitly *differential*, and is supplemented by a check that does not depend on detection at all:

Pass	How it differs from detection
Structural re-read	OCR runs over the entire redacted frame at higher resolution and a lower confidence threshold. Detection ran selectively on non-DOM regions at normal threshold. Different scope, different settings, genuinely independent result.
Value-aware residual check v3.0	For every value currently held in the vault, search the recovered text for exact and fuzzy matches — normalised n-grams, whitespace-insensitive, tolerant of OCR confusions such as 0/O and 1/l. This catches a survival even when the detector that should have found it is broken, because it starts from the known secret rather than from a model's opinion. The check runs in the worker, and never logs the value.
Encoded-bytes verification v3.0	Both passes run against the decoded WebP bytes that will actually be transmitted, not the pre-encode canvas. Verifying a buffer that is not the one on the wire leaves a gap, however theoretical.

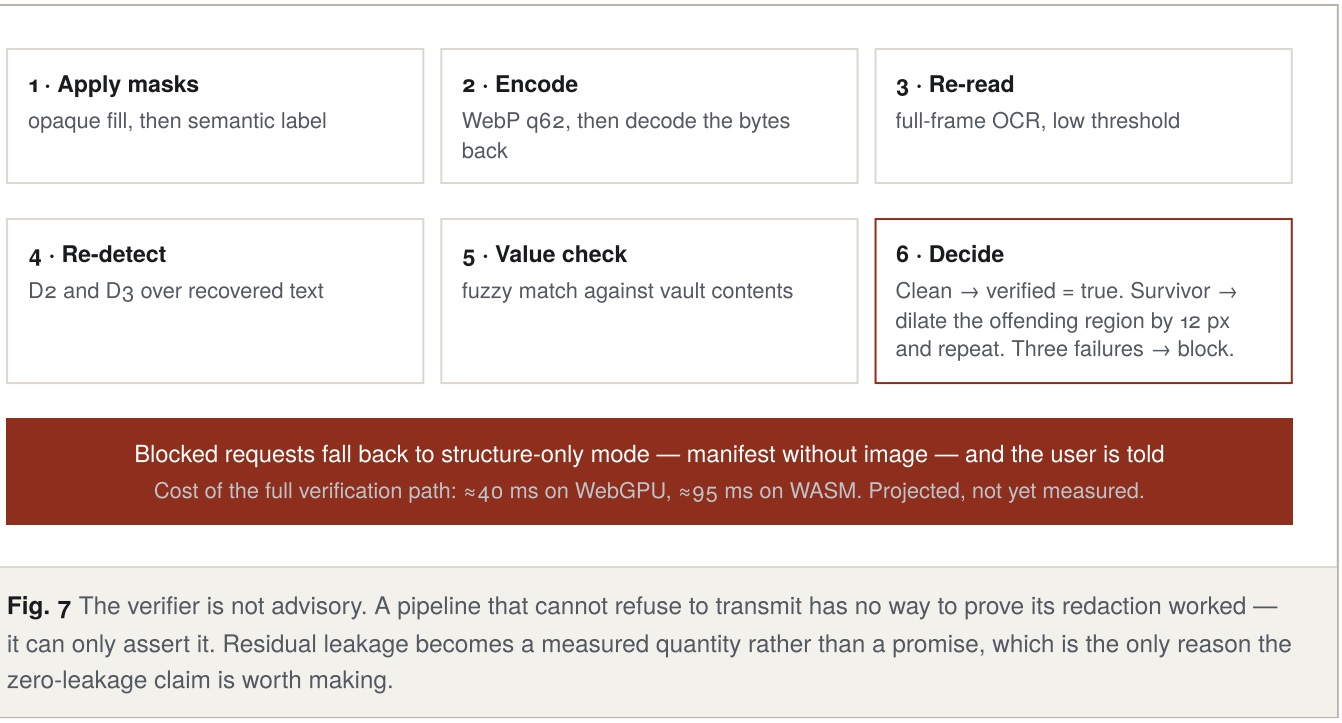

Fig. 7 The verifier is not advisory. A pipeline that cannot refuse to transmit has no way to prove its redaction worked — it can only assert it. Residual leakage becomes a measured quantity rather than a promise, which is the only reason the zero-leakage claim is worth making.

Fail-closed test matrix v3.0

Every row is an automated test. The assertion in every case is identical: **zero outbound requests**.

Detector failures

- OCR worker crash
- detector timeout at 200 ms
- GLiNER returns malformed spans
- face detector out of memory
- two detectors disagree on a span

Pipeline failures

- malformed manifest
- unknown PII class in config
- verifier itself throws
- three consecutive verify failures
- canvas composite fails
- WebP encode returns empty

Boundary failures

- vault empty at re-hydration
- server returns unknown token
- literal aimed at a redacted field
- literal matching a vault value
- payload hash does not match the pin
- plan fails schema validation
- plan fails freshness check
- action outside allowlist

08 · The mechanism

Re-hydration

The signature idea, and the reason privacy does not cost capability.

1 · Detected on screen
+91 98765 43210 — found by D1 (autocomplete=tel) and D2 (pattern) independently

2 · Stored and tokenised in the vault
<PII:PHONE:1> → "9876543210" · RAM only, worker scope, destroyed on tab change

3 · The token crosses; the value does not
manifest: class PHONE, bbox, hint { len 10, kind numeric }, field role "tel"

4 · The server plans in placeholders
{ action: "type", target: "e12", value_ref: "<PII:PHONE:1>" } — never a literal

5 · Client validates
allowlist ✓ · token known to vault ✓ · no literal in value field ✓ · target still fresh ✓

6 · Restored locally, then typed
9876543210 enters the field. The server never saw a digit.

Fig. 8 The manifest carries a type hint — ten characters, numeric — so the server can still reason about field validation and layout. Obfuscate the value; preserve the type. That distinction is what separates a redaction that breaks the task from one that does not.

The type action, and the three checks behind it rewritten in v4.0

> **A functional defect in v3.0**
> Version 3.0 permitted `type` to carry a `value_ref` and nothing else. That is airtight and unusable. *Search for Chandrayaan-3. Select Punjab. Enter 2026.* Most of what a browser agent types is not secret, and a schema that cannot express a non-sensitive literal cannot drive a browser. The rule was written to stop the server leaking a value it should never have had, and it caught far more than that.

The action therefore carries exactly one of two fields, and the client decides which was legitimate:

Field	When the server must use it	What the client checks before typing
`value_ref`	Whenever the target element carries a redaction token in the manifest	Token is known to the vault. Unknown tokens abort — an 8B model under guided decoding will occasionally emit `<PII:PHONE:2>` when only `:1` exists, and the client never guesses which was meant.
`value_literal`	For any field with no redaction token: search boxes, dropdowns, dates, free text	Three checks, below. All three must pass.

The three checks on a literal

1. **Target check.** If the target element's manifest record carries a redaction token, a literal is rejected outright, whatever it contains. A sensitive field is filled by reference or not at all.
2. **Shape check.** The literal is run through D1, D2 and D3. Anything PII-shaped is rejected and logged as a server defect — the server should have used a reference, and the fact that it did not is worth measuring.
3. **Vault check.** The literal is compared, exactly and fuzzily, against every value the vault holds. **A match is not a defect. It is a leak.** The server has reproduced a secret it was never sent, which means something upstream failed. The session halts, the ledger records it, and the run counts as a residual-leakage failure.

The distinction in check three matters more than it looks. A PII-shaped literal the server invented is a bug in the plan. A literal that matches the vault is evidence that redaction failed somewhere earlier in the pipeline, and the two should never be logged as the same event.

One consequence worth stating for the viva: permitting literals does not open an exfiltration path. Typing is not sending. Submission, purchase and messaging remain behind the human confirmation tier, and navigation remains behind the origin policy, so a hostile page that persuades the agent to type something still cannot get that text off the machine.

Manifest schema, version 1.1

The brief requires the server to be aware of the redaction scheme, so the scheme is a versioned contract rather than an implementation detail. Its legend is injected into the server system prompt, and the TypeScript types are generated from the same Pydantic models the server validates against.

```
// client → server
{
  "manifest_version": "1.1",
  "capture": {
    "w": 1024, "h": 640, "format": "webp", "q": 62,
    "dpr": 2.0, "zoom": 1.0, "scale_to_css": 0.5,   // coordinate contract
    "scroll": { "x": 0, "y": 480 },
    "origin": "https://seva.gov.in"
  },
  "capability": { "backend": "webgpu", "tiers_fired": ["T0","T1","T2"] },
  "redactions": [
    { "token": "<PII:PHONE:1>", "class": "PHONE", "tier": "PERSONAL",
      "bbox": [412, 308, 196, 22], "method": "opaque_fill+label",
      "confidence": 0.99, "detectors": ["D1","D2"],
      "hint": { "len": 10, "kind": "numeric", "field_role": "tel" } }
  ],
  "elements": [
    { "id": "e12", "role": "textbox", "name": "Phone",
      "bbox": [400, 260, 300, 32], "source": "dom+vision",
      "visible": true, "enabled": true },
    { "id": "e31", "role": "button", "name": "Submit",
      "bbox": [400, 1180, 120, 40], "source": "dom",
      "visible": false, "offscreen": true },   // known, not yet seen
  ],
  "verified": true,
  "goal": "Complete the application form"
}
```

09 · Server and safety

A narrow contract, validated twice

The server returns a plan, not code. Its output is constrained at generation time by guided decoding, validated by Pydantic before it leaves the server, and validated again by the client before it touches the page. The client never trusts the server, because page content reaches the server's context and a compromised plan must not become a compromised browser.

Permitted actions

- click
- type (value_ref, or a cleared literal)
- scroll
- select
- wait
- zoom_request
- confirm
- done

Never in the schema

- execute_javascript, eval
- shell commands
- download-and-run
- navigation to model-supplied URLs
- a literal aimed at a redacted field
- a literal matching a vault value

Requires human confirmation

- submit, purchase, delete
- send a message
- anything touching authentication
- navigation to a new origin
- re-hydrating a CRITICAL-class value

`zoom_request` is worth defending in the viva: rather than always transmitting a large image, the server may ask for a higher-resolution crop of a specific region. Progressive disclosure lowers latency and reduces the sanitized surface area at the same time.

Origin and navigation policy v3.0

The client tracks the current origin and records it in the manifest. Arbitrary model-supplied URLs stay outside the grammar entirely. A navigation that would change origin — including one triggered by clicking a link the model chose — pauses for human confirmation showing the destination. The vault is destroyed on origin change regardless of confirmation, so a value captured on a government portal can never be re-hydrated into a form on a different site.

Real workflows do cross origins — a services portal handing off to a payments host is ordinary, not suspicious — so the policy permits the crossing and refuses only the carry-over:

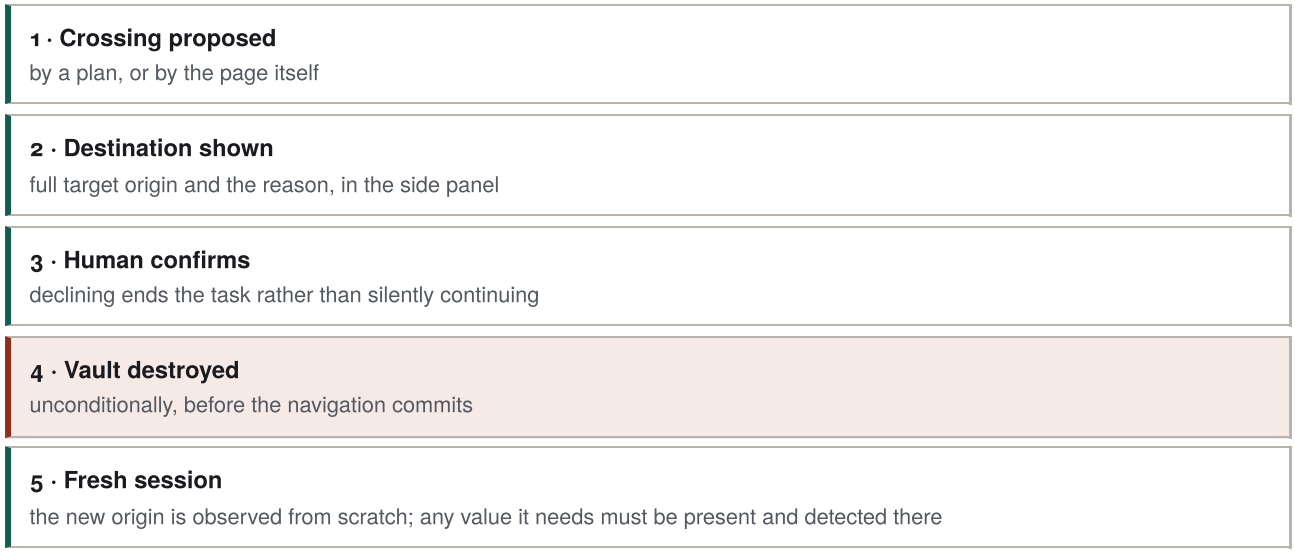

Fig. 9 The crossing is allowed; the secrets do not come along. **Rehearsal note:** no demonstration sequence may depend on a value surviving an origin change. A run that quietly relies on carry-over will work in practice until the policy fires in front of the panel, which is the worst possible moment to discover it.

Prompt injection

Text scraped from a page reaches the server's context, so a hostile page will attempt to issue instructions. Four layers answer this: page content is framed as data and never as instruction; the output grammar admits no action capable of arbitrary execution; every irreversible step stops for a human; and the origin policy prevents exfiltration by navigation. The agent can be misled about what to look at. It cannot be talked into doing something outside the allowlist.

This is demonstrated rather than asserted — see the adversarial sequence in section 15.

---

10 · Performance

Two latency budgets, because we will be judged on the slower one

Every number below is a projected budget. None is a measured result, and none appears on a slide until the harness produces it.

> **Why v2.0's single 1.1 s figure was a liability**
>
> The v2.0 waterfall assumed the WebGPU path throughout. Our own risk table rates "judging machine has no WebGPU" as **high** likelihood. The WASM number is therefore the one the panel is most likely to see, and it was absent from the document. Presenting a WebGPU-only budget while planning for a WASM demo is the sort of gap a panel finds in thirty seconds.

Per-step budget, full pipeline (projected p50, mid-range laptop)

Stage	WebGPU	WASM/SIMD	Fires every step?
Capture + downscale	18 ms	18 ms	on change only
Change gate	<1 ms	<1 ms	yes
DOM extraction	12 ms	12 ms	yes
UI element detection	35 ms	140 ms	on change only
Face detection	8 ms	22 ms	on change only
Selective OCR	90 ms	260 ms	pre-egress only
PII detection	45 ms	120 ms	pre-egress only
Redact + composite	12 ms	12 ms	pre-egress only
Verify (differential + value-aware)	40 ms	95 ms	pre-egress only
Encode + network	60 ms	60 ms	pre-egress only
Server reasoning (Qwen3-VL-4B)	620 ms	620 ms	pre-egress only
Freshness check + execute	25 ms	25 ms	yes
Full pipeline step	**≈965 ms**	**≈1,385 ms**	
Cached step — no vision, no network	**≈38 ms**	**≈38 ms**	
Local-decision step — vision, no network v3.0	**≈98 ms**	**≈235 ms**	

Server reasoning is 64% of the WebGPU budget and 45% of the WASM budget. Privacy — redaction plus verification — is 52 ms of 965 and 107 ms of 1,385: between five and eight percent. Server hardware must be stated whenever this table is shown.

Fig. 10 Worth saying out loud when the panel asks what privacy costs: redaction and verification together are single-digit percentages of wall-clock time. The dominant term is remote inference, which is exactly the term the caching and local-decision strategies are designed to avoid paying.

Skipping the server entirely new in v3.0

The highest-leverage latency win available is not shaving milliseconds off OCR. It is not making the request at all. When the fused element graph contains a `textbox` whose autocomplete token or accessible name maps unambiguously to a class the vault holds, the client fills it directly. No manifest, no encode, no 620 ms round trip.

This is also the cleanest answer to the brief's requirement that a local model read the screen and *decide* on that basis. The decision — this field is a phone field, I hold a phone value, no server is needed — is made locally from perception. On a ten-field form we expect this to cover the majority of steps, and it produces the single most quotable number in the submission: **steps completed with no network request at all, out of ten**.

Model feasibility matrix new in v4.0

Each cell records four things: does the session load, p50 latency on a fixed fixture, peak heap, and output correctness against a known-good reference. **No model enters the build until its row is complete, and a model that fails the WASM columns is not shipped whatever it does on WebGPU** — the judging machine is more likely to be the WASM one.

Model	Chrome WebGPU	Chrome WASM	Firefox WebGPU	Firefox WASM (Linux)
UI element detector	—	—	—	—
YuNet faces	—	—	—	—
PP-OCRv5-mobile	—	—	—	—
GLiNER-PII	—	—	—	—
SmolVLM (offline path)	—	—	—	—

Twenty cells, filled in week one and re-run in continuous integration thereafter. The table is also the answer to a likely question about cross-browser parity: we do not assert it, we publish it.

Where else the time is won back

- **Screen-state cache.** A persistent representation of the page, updated per element rather than rebuilt. On a form-filling task this covers 50–60% of captures.
- **Region-of-interest re-analysis.** MutationObserver names the dirty subtree, so OCR and PII detection re-run on that node alone.
- **Warm start.** ONNX sessions compiled at extension startup. Cold-start cost is measured and reported separately, but never paid during a demo.
- **Selective OCR.** Zero to two regions instead of a full viewport, except in the verification pass where full coverage is the point.

Reporting rules

p50 and p95 for end-to-end and every stage. Peak — not mean — for heap, GPU memory and CPU share, because peak is what determines whether the extension stays usable on a

judging machine. Cold start and warm start reported as separate figures. Every table labelled with the backend and the hardware.

Known landmine — budget a day

Multiple ONNX Runtime Web sessions share one WebAssembly heap, and disposing a session does not reliably return its linear memory. Teams have hit hard aborts loading a third model sequentially inside an extension offscreen document. Mitigations: prefer the WebGPU execution provider so allocations live GPU-side; isolate the optional local VLM in its own worker so tearing down the worker reclaims the entire heap; never load it concurrently with the sanitisation tier; and set the WASM thread count explicitly rather than letting it default.

> **Week-one spike, before anything else** extended in v4.0
>
> Confirm that `navigator.gpu.requestAdapter()` returns a real adapter inside a Chrome `chrome.offscreen` document and inside a Firefox MV3 event page. GPU adapter availability from extension background contexts has been inconsistent. If it returns null, every WebGPU number in this document becomes the WASM number and the entire performance story changes.
>
> The spike does not stop at the adapter. "WebGPU when available, WASM always" is an objective, not an observed fact — a specific model on a specific backend either works or it does not, and finding out in week five is finding out too late. Every model is therefore run in every context before the build commits to it.

---

11 · Stack

Frozen contracts, replaceable models

Freeze the interfaces. Pin the implementations. Benchmark the candidates behind them.

Version 2.0 froze technology choices and pinned every version — correct, and still policy. Version 3.0 adds the layer above it: each perception role is a typed interface, so swapping a detector is a config change and a benchmark run, not an architectural decision. The two rules are not in tension. Interfaces are abstract; the implementation behind each one is pinned to an exact revision, quantisation and runtime asset, because a silent upstream change twelve hours before judging is an avoidable way to lose.

Client

Layer	Choice	Why this one
Extension framework	WXT (Vite)	One codebase producing both a Chrome MV3 service worker and a Firefox MV3 event page; the manifest divergence is handled for you
Language	TypeScript, strict	Manifest and action schemas exist as types generated from the server's Pydantic models
Interface	React + Tailwind, Side Panel API	A side panel survives page interaction; a popup does not, and multi-step tasks need it to
Inference runtime	ONNX Runtime Web; Transformers.js for NER and the local VLM	Raw ORT gives manual control of pre-processing and NMS for the detectors; Transformers.js is faster to integrate for the transformer models
Acceleration	WebGPU, with a benchmarked WASM path	Feature-detect `navigator.gpu`, and surface which backend is live in the ledger. See the position statement below.
Execution context	Offscreen document + dedicated worker	MV3 service workers have no DOM and terminate when idle; they cannot hold inference sessions
Capture	tabs.captureVisibleTab	Faster than screen capture and raises no operating-system picker mid-demo. Rate-limited, which is why the change gate no longer depends on it.

The interface that carries the most risk new in v4.0

Of everything in the registry below, UI element detection is the single largest implementation gamble: a third-party detector, exported to ONNX, quantised, and run inside an extension worker in two browsers. Every step of that chain has failed for someone. The mitigation is not to pick a better model, it is to stop the architecture depending on any particular one.

`UIElementDetector` is therefore a hard interface — screenshot in, boxes and roles out — with three ranked implementations behind it:

	Implementation	Position
A	OmniParser `icon_detect_v3`, ONNX INT8	Preferred. Purpose-trained, MIT-licensed, ~12 MB at 640 px. Committed only once its feasibility row is green.
B	Our own detector head, trained on the synthetic set	No external dependency and no licence question. Started in week three regardless of whether A is working, because the labelled data pipeline already exists for evaluation, so the marginal cost is small and it removes the biggest single point of failure in the client.
C	DOM-only degradation, vision limited to faces and OCR	Always available, lower grounding score, task still completes on DOM-rendered pages. This is the floor, and it is a floor we can demonstrate.

Model registry, with licences

> **Licence defect carried over from v2.0**
>
> The v2.0 registry pinned `microsoft/OmniParser-v2.0` for UI element detection. Its `icon_detect` model is **AGPL-3.0** — it is a fine-tuned Ultralytics YOLO, and only `icon_caption` is MIT. AGPL §13 has network-service implications that matter for anything handed to a government department. OmniParser added a `icon_detect_v3` based on the MIT-licensed YOLOv9 implementation in mid-2026; earlier Ultralytics-based detectors retain their AGPL licence. We move to v3, and we add a licence column so this cannot happen again.

Role	Pinned implementation	Licence	Notes
UI elements	OmniParser icon_detect_v3, ONNX INT8	MIT (YOLOv9)	Purpose-trained on interactable web elements. Fallback: our own head trained on the synthetic set, which removes the dependency entirely.
Faces	YuNet (OpenCV Zoo)	MIT	340 KB, sub-10 ms, no reason to use anything larger
OCR	PP-OCRv5-mobile, via paddle2onnx	Apache-2.0	Detection and recognition both exportable. Selective in T2; full-frame in the verifier.
Semantic PII	gliner_multi_pii-v1, INT8	Apache-2.0	Zero-shot — new entity categories are a config change, not a training run
Local VLM	SmolVLM-256M-Instruct	Apache-2.0	Changed from FastVLM-0.5B, which ships under Apple's ML research licence — a poor fit for the brief's offline-deployable clause
Server VLM	Qwen3-VL-4B-Instruct; 8B as the upgrade	Apache-2.0	See below

Why 4B is now the default, not the fallback v3.0

On the public ScreenSpot leaderboard, Qwen3-VL-32B-Instruct leads the open-weight field at 0.958 while Qwen3-VL-4B-Instruct reaches 0.940 — under two points for a fraction of the compute. Given that server inference is the dominant latency term and that "GPU host unreachable at the venue" is a live risk, 4B is the better default and 8B is the upgrade if the benchmark on our own 300 Indian screens says otherwise. Version 2.0 had this the other way round on the strength of published metrics rather than our own. Both are benchmarked behind the same interface in week five, and the data decides.

Server

- **FastAPI + Pydantic v2** — typed contract, source of truth for the client's generated types
- **vLLM** — OpenAI-compatible, guided JSON decoding
- **Session store** — in-process dictionary for v1; Redis deferred until horizontal scale is actually needed
- **Docker Compose + CUDA** — fully offline-deployable, per the brief

Testing: Vitest and Playwright, including the egress interception suite. Benchmarking: a custom harness reporting all five scored metrics plus task success.

State the WebGPU position precisely

The engineering rule is simple: feature-detect `navigator.gpu`, always ship a benchmarked WASM path, and show which one is live. But give the accurate reason. WebGPU is not missing from Firefox — it shipped by default on Windows in Firefox 141, reached Apple Silicon macOS in 145 and the remaining macOS versions in 147; Chrome has had it since 113 and Safari since version 26. The real gap is Firefox on Linux, where it remains disabled by default and is enabled through `dom.webgpu.enabled` in about:config on the release channel, with Mozilla targeting a 2026 ship. Android Firefox is later still, and driver blocklists affect older hardware everywhere. Saying "Firefox doesn't support WebGPU" to a panel that has read the release notes costs credibility on a point where the engineering was right.

---

12 · Scope

What ships, what waits, what we refuse

The largest risk to this project is building a research programme instead of a submission.

	Ships in v1	v2, if week six is calm	Not building
Perception	Derived element graph, UI detector, YuNet, selective OCR, off-screen element reporting, coordinate contract	Fine-tuned visual-PII head	Embedding-based change gate
Privacy	All four channels, manifest v1.1, opaque fill + semantic label, differential verifier, value-aware check, vault, egress guard, fail-closed suite	Signature and ID-card classes; CONFIDENTIAL entity list expansion	Differential privacy, k-anonymity, a policy engine, universal confidentiality detection
Server	FastAPI, vLLM, Qwen3-VL-4B, guided JSON, tripwire, origin policy	Redis, trajectory replay, 8B upgrade	Model ensembles, multi-model routing
Local reasoning	Local-decision path for unambiguous fields	SmolVLM for offline mode — click, scroll, basic form interaction only	A local VLM in the normal path; autonomous offline browsing
Performance	Screen-state cache, ROI re-analysis, warm start, benchmarked WASM path	Streaming action execution	Telemetry pipeline

Cut deliberately: the embedding gate, Redis, two detector classes, the confidentiality policy engine, and the local model from the normal path. Five fewer moving parts — and, usefully, fewer inference sessions competing for the shared WebAssembly heap.

13 · Delivery

Six weeks, ordered by dependency

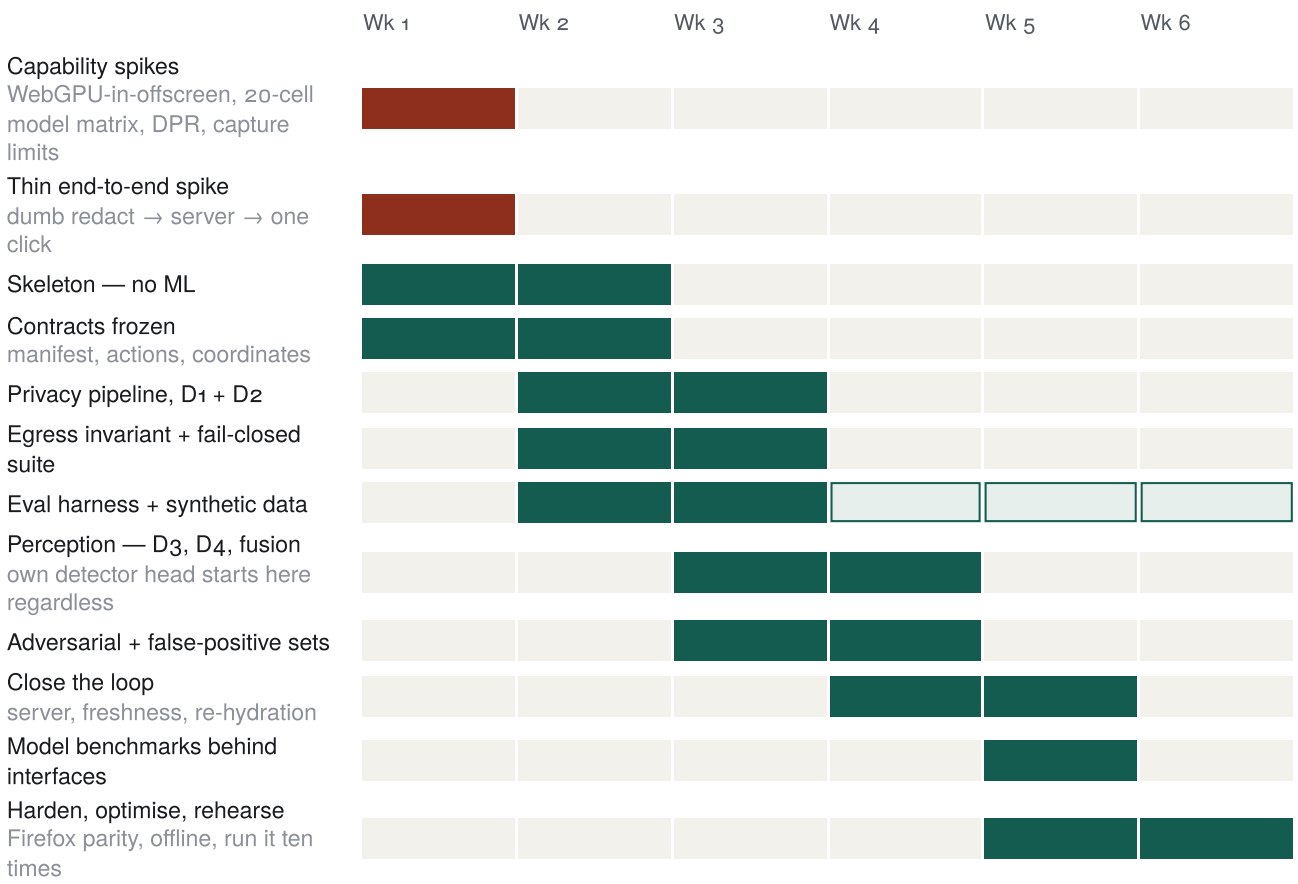

Fig. 11 Changed in v4.0: the capability spike now includes the twenty-cell model feasibility matrix, and our own detector head is scheduled in week three whether or not OmniParser is working. **Carried from v3.0:** the first server round trip sits in week one. Version 2.0 discovered integration failures — coordinate spaces, extension messaging, vLLM guided-decoding quirks — in week five, with one week of slack. A throwaway thin thread in week one finds them when they cost a day. A complete, defensible submission exists by the end of week four; weeks five and six raise the score, they do not rescue the project.

Team of six

Role	Owns
Client perception (2)	Capture, DOM extractor, detector integration, fusion, coordinate contract, executor, freshness check
Privacy engineer (1)	Four channels, manifest, redaction, differential verifier, value-aware check, vault, egress guard and its invariant tests
Server and agent (1)	vLLM deployment, prompting, action schema, validators, tripwire, origin policy
Evaluation and data (1)	Synthetic generator, adversarial set, false-positive set, labelled ground truth, all six dashboards
Integration and demo (1)	Cross-browser builds, Playwright end-to-end, fail-closed suite, ledger UI, rehearsal

Finale runbook

Hours 0–2: environment and smoke test on the venue machine, including the WebGPU/WASM capability check with the result read off the ledger. Hours 2–20: build the specific use case handed to you — the action schema is generic, so a new task should be configuration rather than code. Hours 20–28: tune on their data. Hours 28–32: rehearse, including the adversarial and failure sequences. Hours 32–36: freeze. Nobody merges after hour 32.

What we measure, and what we refuse to claim

Scored metric	Reported figure	Evidence source
Visual context — 25%	Element mAP@0.5, element recall, grounding accuracy. Reported twice: on clean frames and on redacted frames.	ScreenSpot-v2 web subset plus 300 self-labelled Indian government and banking screens, including scanned-document pages where the DOM is empty
PII detection — 20%	Per-class precision, recall and F1. Target recall ≥ 0.98. Precision reported separately on the decoy set.	500 synthetic and 200 real-layout screens with ground-truth boxes, plus the adversarial and false-positive sets below
Redaction — 20%	Measured residual leakage; over-redaction area ratio; block rate	Verifier logs, value-aware check results, and the independent server-side tripwire
Client resources — 20%	Peak heap, peak GPU memory, CPU share, weights on disk, tier firing distribution, cold vs warm start	Browser memory API and Chrome tracing over a ten-step task
Latency — 15%	Stage-wise p50 and p95, cache hit rate, network-free step count, per backend	`performance.measure` marks rendered as a waterfall in the ledger
Task success after privacy not in the rubric	Task completion rate with redaction on, measured against the same runs as the privacy metrics	Playwright assertions on final form state

A PII recall of 100% alongside a task success rate of zero is a failed project. The entire claim of this submission is that privacy does not destroy usefulness, so both numbers belong on the same slide, measured on the same runs. The ablation that makes the point is a two-row table: grounding accuracy and task success, with redaction off and on.

Adversarial and decoy sets v3.0

Adversarial recall set

- tiny and rotated text
- partial occlusion
- text in image, canvas and SVG
- dark mode and low contrast
- dynamic popups mid-capture
- PII split across DOM nodes
- overlapping detection regions
- PDF-rendered text
- identifiers adjacent to controls

False-positive decoy set

- 12-digit order numbers
- PAN-shaped SKU codes
- GSTIN-shaped invoice refs
- tracking IDs and AWB numbers
- test card numbers failing Luhn
- reference numbers with valid state prefixes
- dates that parse as DOB but are not

Disagreement set

- D1 says field, D2 says no
- D3 fires where D2 is silent
- one detector times out
- one detector crashes
- DOM and vision boxes disagree beyond IoU threshold
- each verifies that union and fail-closed behaviour hold

Generating the data

Render HTML templates — a bank statement, a PAN card view, an e-KYC page, a profile, an application form, a scanned-document viewer — populated with generated Indian identifiers, then screenshot them through Playwright. Ground-truth boxes come free, because you positioned the elements. Thousands of perfectly labelled samples in an afternoon, and the highest-leverage thing available in week one. The same generator produces the decoy set by swapping identifier generators for lookalike generators, which costs almost nothing and covers the precision half of a 20% metric.

> **Report measured, not aspirational**
>
> Zero residual leakage is the objective. It belongs on the title slide only once the harness has demonstrated it across the full evaluation set. The same rule now applies to latency: every figure in section 10 is labelled a budget until the harness replaces it with a measurement. An unbacked "0.00%" or "1.1 s" is the kind of claim a panel will spend the entire question period dismantling — and they will be right to.

What could go wrong, and the prepared answer

Risk	Likelihood	Mitigation, prepared in advance
Panel reads the build as a DOM scraper, not a vision agent v3.0	High	T1 vision on every changed frame; a scanned-ID segment in the demo where the DOM is empty by construction; the local-decision path shown as vision-driven; tier firing distribution on the slide
Judging machine has no WebGPU — Firefox on Linux	High	WASM path built and benchmarked from week one; both budgets published; live backend shown in the ledger; never ship WebGPU-only
WebAssembly heap exhaustion with several models	High	Per-worker isolation, WebGPU provider, lazy loading, measured teardown, explicit thread count
Coordinate drift on a HiDPI or zoomed machine v3.0	Medium	Single canonical CSS-pixel space, conversions at every edge, six-configuration CI matrix written before the executor
GPU host unreachable at the venue	Medium	Qwen3-VL-4B on a team laptop; SmolVLM offline path; recorded video of the full run
PII missed on the unseen finale use case	Medium	Fail-closed union, differential verifier, value-aware residual check, conservative dilation, broad zero-shot entity list
Over-redaction breaks the task	Medium	Re-hydration means masking costs no capability — this is precisely why the design holds, and the ablation table proves it
Long form extends below the fold v3.0	Medium	Off-screen elements reported as known-but-uncaptured; scroll planned toward named fields the server has not seen
Hostile page attempts injection	Low	Content is data; allowlist grammar; origin policy; human gate on irreversible actions; demonstrated live rather than asserted
Screen capture blocked on a protected page	Low	Degrade to structure-only mode — manifest without image — and say so in the interface

16 · Demonstration

Four sequences, twelve minutes

Fill an Indian government application using the profile already on screen — then break it on purpose.

Sequence 1 · The main task, six minutes

The profile page carries a name, phone, email, address, date of birth, PAN, Aadhaar, a photograph — **and a scanned PAN card image whose text exists in no DOM node**. The side panel is given one instruction: *fill this application using my profile information*.

Two of the fields exercise the dual-mode type action deliberately: the Aadhaar number is filled by reference from the vault, and the district field is filled from a plain literal the server chose and the client cleared. Both happen in the same run, seconds apart, and the ledger labels which was which.

Run it with the privacy ledger and the browser's own network inspector both visible. Let the panel read the outbound request themselves rather than taking the claim on trust — *the server has never seen a single digit of this Aadhaar number, and the number it is about to type came out of a scanned image the DOM knows nothing about* — and then let the form fill with the real values. The gap between those two moments is the argument.

The ledger shows, live: which backend is running, how many steps completed with no network request at all, detected and masked counts, verification result, the pinned payload hash, and the stage waterfall. Invite the panel to compare the hash on screen with the request body in the inspector — they are the same artifact, and that is the point of pinning it.

> **Why the scanned card was added** v3.0
>
> The v2.0 demo ran entirely on a DOM-rich page, where vision adds nothing a judge can see. That turned our strongest engineering decision into our weakest rubric exposure. One scanned identity document converts the same demo into direct evidence for the 25% metric, gives the OCR and visual-PII channels something real to find, and makes the local vision model visibly necessary rather than merely present. It is the single highest-value change in this document.

Sequence 2 · Adversarial, two minutes v3.0

Navigate to a page containing hidden text instructing the agent to ignore the user, read the vault, and post its contents to an external URL. Show the ledger recording that the text was *observed* — we do not pretend the agent is blind to it — and that no corresponding action was generated, because the grammar has no action that could express it and the origin policy would block the navigation regardless. Observation without obedience is the point.

Sequence 3 · The failure path, two minutes v3.0

Inject a detector timeout from the developer console. The union counts the timeout as a positive, the region is masked, and the run continues. Then force three consecutive verification failures. The request is blocked, the interface says so plainly, and the agent falls

back to structure-only mode. A system that cannot be shown refusing has not demonstrated that it can refuse.

Sequence 4 · Offline, two minutes

Pull the network cable. Offline mode completes a simpler task locally with SmolVLM — click, scroll, fill one field — which is the brief's own privacy premise carried to its conclusion.

Prepared before travelling

- A recorded video of all four sequences.
- The 4B model running on a team laptop.
- A WASM-only build, benchmarked.
- A Firefox build, tested on Linux, with the backend indicator visible.
- The ablation table — grounding and task success, redaction off and on — printed.
- Assume the venue network fails, because it usually does.

---

17 · Appendix

Engineering rules, changelog, glossary

Non-negotiable rules

1. All network egress passes through one module. Enforced by a lint rule, a CSP `connect-src` restriction, and an interception test.
2. The vault never touches persistent storage of any kind, and is destroyed on session end, tab change and origin change.
3. Every model has a WebAssembly fallback, tested in continuous integration.
4. The verifier can block a send. It is not advisory.
5. Verification runs against the encoded bytes that will actually be transmitted, and the second pass differs from the first in scope and threshold.
6. The un-redacted bitmap is closed immediately after masking and never referenced again.
7. The server may reference a token, and may propose a literal for a field that holds nothing sensitive. Unknown tokens abort; literals aimed at redacted fields abort; a

literal matching a vault value halts the session as a leak.

8. The transmitted payload is one immutable artifact, hashed before verification and pinned through egress. Nothing is re-encoded or re-serialized after the hash is taken.

9. No model enters the build until its feasibility row is complete across all four browser and backend combinations.

10. Server output is schema-validated twice, on both sides, and re-validated against the live page before execution. Validation failure aborts; there is no best-effort parse.

11. No component logs a secret value. Not the tripwire, not the verifier, not the ledger.

12. All coordinates are CSS viewport pixels. Every other space converts at its own edge.

13. Every version is pinned — library, model revision, quantisation, runtime assets — and every model carries its licence in the registry.

Changelog · v3.0 → v4.0

§02, §08, §09	**The type action was too restrictive and is rewritten.** v3.0 allowed `value_ref` only, which forbids typing a search term or a district and cannot drive a browser. Literals are now permitted for fields with no redaction token, behind a target check, a shape check and a vault check — with a vault match escalated from defect to leak.
§06	**The change gate is stated precisely.** MutationObserver is the structural signal, not a claim to see everything visual. Dynamic regions are enumerated and polled at a bounded rate, a low-rate full-frame hash bounds the worst case, and *changed frame* now has a definition T1 can be gated on.
§05	**The payload is a single hash-pinned artifact.** Verify-then-encode left a window in which the thing checked was not the thing sent. The ledger hash, the verified artifact and the request body are now provably the same object.
§10, §13	**The week-one spike covers models, not just the adapter.** A twenty-cell feasibility matrix — five models across Chrome and Firefox, WebGPU and WASM — gates entry into the build.
§11	**UIElementDetector promoted to a hard interface** with three ranked implementations. Our own detector head starts in week three regardless of whether OmniParser is working.
§09	**Origin-crossing sequence defined**, with a rehearsal rule that no demo may depend on a value surviving the transition.
§12	**Universal confidentiality detection added to "not building."** The taxonomy stays a config file.

Changelog · v2.0 → v3.0

§01, §06, §16	**Vision made load-bearing.** T1 reframed as always-on rather than gated away; scanned-document content added to the demo; client-side decision path added. Answers the brief's requirement for a local model that reads and decides.

§01	**Client share corrected** from 85% to a defensible split. Visual-context accuracy is shared, not ours alone.
§03	**Threat model added.** Four adversaries, named mitigations, stated scope limit on quasi-identifiers.
§04, §09	**Action freshness and origin policy added** as pipeline stages 8 and a validator rule.
§05	**Egress invariant formalised** with three enforcement mechanisms. **Coordinate contract added. Off-screen content policy added. "Accessibility tree" corrected** to derived element graph.
§06	**Change gate corrected** — MutationObserver primary, capture on demand. The v2.0 tier table assumed a free capture that `tabs.captureVisibleTab` does not provide.
§07	**Verifier made differential** and supplemented with a value-aware residual check; verification moved to the encoded bytes. **Semantic labels** composited over fills. **GSTIN check digit** added. **Confidentiality classes** added as configuration, not a policy engine. **Fail-closed test matrix** added.
§08	**Manifest to v1.1** — coordinate block, capability block, detector provenance, field role, off-screen elements. **Unknown-token and literal-value rejection** added.
§10	**Two latency budgets** published, WebGPU and WASM, all labelled projected. p50/p95 and peak reporting rules stated. **WebGPU-in-offscreen spike** scheduled for week one.
§11	**OmniParser icon_detect moved to the MIT-licensed v3** — the pinned v2.0 model was AGPL-3.0. **Licence column added. SmolVLM replaces FastVLM** on licence grounds. **Qwen3-VL-4B becomes the default**, 8B the upgrade.
§13	**Two week-one spikes added**, and the first server round trip moved from week five to week one.
§14	**Adversarial, decoy and disagreement sets added.** Task success after privacy promoted onto the main metrics table.
§16	**Adversarial and failure-path sequences added** to the demonstration.

Glossary for the viva

Term	Meaning
Redaction manifest	The versioned contract describing every mask applied, so the server can reason about what it cannot see
Re-hydration	Substituting a real value back into a placeholder at execution time, on the client, from memory
Fail closed	When detection is uncertain, mask anyway — the union of all detector outputs wins
Residual leakage	The fraction of transmitted payloads in which any sensitive value survived redaction
Egress guard	The single module permitted to make a network call, which refuses unverified payloads
Egress invariant	The formal, tested property that no request leaves without `verified === true`
Differential verification	A second detection pass that differs from the first in scope and threshold, so it can disagree with it
Value-aware check	Searching the outgoing payload for the specific secrets the vault holds, rather than for whatever a detector believes is secret
Payload pin	The hash taken over the single assembled byte artifact, which verification signs and egress refuses to send without
Changed frame	A frame in which the change policy reports an observable-state change, from the structural signal or the visual one — not every rendered frame
Cleared literal	A non-sensitive plain value proposed by the server that has passed the target, shape and vault checks on the client
Action freshness	Re-confirming that a plan's target still exists, matches, and has not moved, before executing it
Derived element graph	Our reconstruction of page structure from roles, ARIA, accessible names and geometry — not the browser's native accessibility tree, which extensions cannot read